Project Proposal

Predicting Road Accident Severity in UK

A Multi-Source Machine Learning Approach

Course: Project in Artificial Intelligence (02360502)

Semester: Spring 2026

Student 1: Ibraheem Akaree | ID: 212930846 | Email: ibraheema@campus.technion.ac.il

Student 2: Sewar Abahry | ID: 325345734 | Email: sewar.abahry@campus.technion.ac.il

Advisor: Yotam Gavish

1. Problem Description

Road traffic accidents impose a significant burden on emergency services, healthcare systems, and public safety infrastructure. In Great Britain alone, tens of thousands of personal injury accidents are recorded annually by the police. When an accident is reported to emergency services, dispatchers must make rapid decisions — often within seconds and with incomplete information — about the level of response required: a single patrol car, an ambulance, or a full emergency response team.

The central problem addressed in this project is: given only the information available in the first moments after an accident is reported (location, time, road type, weather conditions,…), can we accurately predict the injury severity of the accident?

This framing has direct real-world impact. An accurate severity predictor deployed at 999 dispatch centres could optimise emergency resource allocation, reduce response times for serious accidents, and ultimately save lives.

2. Proposed Solution

We cast the problem as a multiclass classification task. Given a set of features describing the circumstances of an accident at the time of reporting, a trained classifier predicts the severity of the resulting injury.

2.1 Problem Casting

The target variable is derived from the STATS19 accident_severity field. This is formulated as a multiclass classification problem, where the model predicts the severity level of an accident based on the available features. We will use multiple supervised learning algorithms for this task.

2.2 Feature Engineering Strategy

The core creative contribution of this project lies in the feature engineering pipeline. Rather than relying solely on a ready-made feature table, we enrich each accident record with 3 distinct external data sources, each requiring non-trivial joining logic:

- Road infrastructure features extracted from OpenStreetMap via spatial queries
- Annual traffic volume joined by road segment from official DfT (Department for Transport) count data
- Hourly weather measurements at exact accident location and time, via Open-Meteo Historical API

This multi-source enrichment pipeline produces a significantly richer feature space than the base STATS19 data alone, and each data join requires spatial or temporal reasoning that constitutes meaningful algorithmic work.

2.3 Active Feature Acquisition Framework (Reinforcement Learning)

To address the real-world constraints of a 999 dispatch centre—specifically time latency and API usage costs—we extend our system from a static classifier to an **Active Feature Acquisition (AFA)** framework. We formulate this process as a Markov Decision Process (MDP) and solve it using Reinforcement Learning (RL), where an agent sequentially decides which additional piece of information to acquire next — or whether the current level of certainty is already sufficient to make a prediction. Individual STATS19 fields are treated as separate acquisition actions, while each external API source (OSM, AADF, Weather) is a single grouped action, since all features from a given API arrive together in one call.

The State Space

The state at each decision step is defined as $s_t = (x_{base}, x_{acquired}, m_t, p_t)$, representing all available information about a specific accident at a given moment in the episode, where:

- $\boldsymbol{x_{base}}$: The subset of STATS19 features available automatically at the moment of the 999 call (e.g. time). This component is populated before the episode begins and never changes.
- $\boldsymbol{x_{acquired}}$: The partial feature vector, built up incrementally as the agent executes acquisition actions. At $t = 0$ this is empty; each acquisition action appends the corresponding feature(s). Individual STATS19 fields append a single value; external API actions append a group of related features together.
- $\boldsymbol{m_t} \in \{0,1\}^N$: A binary acquisition mask over all N acquirable features and feature groups, indicating which have already been obtained. This prevents re-acquisition and defines which actions remain available at step t.
- $\boldsymbol{p_t}$: The classifier's current predicted probability distribution over the severity classes, computed from the features acquired so far. This vector encodes the agent's current prediction certainty and is updated after every acquisition action. It provides the agent with a direct signal of how much uncertainty remains, enabling it to decide whether further acquisition is worthwhile or whether the current confidence is already sufficient to terminate.

The **initial state** s_0 begins each episode with only x_{base} populated, an empty $x_{acquired}$, mask $m_0 = 0$ (all zeros), and p_0 is calculated based on x_{base} alone. reflecting the real-world scenario where only the automatically available STATS19 fields are known at the moment of the 999 call, and no additional features have yet been acquired.

The Action Space

At each step, the RL agent evaluates the currently available information and chooses from the following discrete set of actions. Individual STATS19 fields each constitute a separate acquisition action; each external API source constitutes a single grouped action (since all features from that API arrive together in one call):

- A_0 **(Predict & Terminate):** Terminate data acquisition and report the current prediction. Since the classifier runs continuously and p□ is already in the state, no additional inference step is needed — A_0 simply signals that the agent is sufficiently confident and the current p□ is the final output.

- A_1, ..., A□ **(Acquire STATS19 Field):** Query the dispatcher for a specific STATS19 field not yet obtained (e.g., number of vehicles involved, road conditions, pedestrian involvement). Each field constitutes a separate action with its own associated cost, reflecting whether the field is known automatically, must be actively asked, or carries

reliability concerns.

- A_{k+1} **(Acquire Road Infrastructure Data):** Query the OpenStreetMap Overpass API to extract localized spatial features (e.g., speed cameras, lane counts, street lighting).

- A_{k+2} **(Acquire Traffic Volume Data):** Perform a spatial nearest-neighbor match to attach the Department for Transport (DfT) Annual Average Daily Flow (AADF) data.

- A_{k+3} **(Acquire Historical Weather Data):** Call the Open-Meteo Historical API to retrieve high-resolution, objective weather measurements for the exact location and hour.

The Transition Function

The transition function defines how the state evolves in response to the agent's actions. During training, all external data is pre-fetched and cached, making the MDP transitions fully deterministic. At inference time, API failures are handled at the engineering level via retry logic and fallback imputation, keeping the MDP formulation clean. The transitions are as follows:

- **If the agent takes an acquisition action A_i**: The corresponding feature(s) are appended to $x_{acquired}$ (a single field value for STATS19 actions, or a full group of features for external API actions), and the corresponding bit of m_t is flipped to 1. The acquisition cost C_i is applied as an immediate negative reward. The classifier for the current external-source subset is then re-run on the updated $x_{acquired}$ and x_{base} to recompute the predicted probability distribution, yielding the updated p_{t+1} for the next state s_{t+1}.

- **If the agent takes the terminal action A_0 (Predict & Terminate)**: The agent terminates. The current predicted probability distribution p_t is selected from the set of 8 pre-trained classifiers (one per possible subset of the three external sources: OSM, AADF, and Weather). Each classifier is trained with random masking of individual STATS19 fields, so it learns to handle any partial combination of STATS19 inputs while knowing exactly which external sources are present. The selected classifier outputs the final severity prediction.

The Reward Function

The objective of the RL agent is to maximise its cumulative reward. Given the variable episode length (up to N acquisition actions plus one terminal action), we use an undiscounted return ($\gamma = 1$). The reward function is designed to encourage the agent to maximise prediction certainty while minimising acquisition cost. It rewards reductions in uncertainty after each feature acquisition, penalises unnecessary acquisitions once confidence is already high, and delivers a strong positive reward for terminating when the model reaches a sufficiently reliable confidence threshold and predicts correctly. The composite reward is formulated as follows:(may be changed in future)

$$Reward = R_{terminal} + \sum_t R_{step}(t)$$

$$R_{step}(t) = \alpha \cdot \Delta Confidence_t - C_i$$

$$R_{terminal} = \beta \cdot \text{Accuracy}(\hat{y}, y)$$

Where:

- $\Delta Confidence_t$: The improvement in prediction confidence after acquiring feature *i* at step *t* (e.g. reduction in entropy of p_t OR how much did the highest class probability increase?). Rewards acquisitions that meaningfully reduce uncertainty.

- $Accuracy(\hat{y}, y)$: Represents the evaluation metric of the final severity prediction (e.g., Macro F1-score) compared against the true ground-truth label (y) from STATS19. This reward component is only delivered at the terminal step (A_0).

- C_i : A strict negative penalty (cost) assigned to each acquisition action i .
  STATS19 field costs vary per feature, reflecting whether the field is known automatically, must be actively asked of the caller, or carries reliability concerns. External API costs are higher, reflecting their latency overhead.

- α, β: Weighting parameters controlling the trade-off between step-level guidance and terminal accuracy. α is kept small relative to β, so that the dense signal guides exploration without overshadowing the final prediction objective. These are treated as design parameters to be tuned empirically and may be revised in future iterations.

Evaluation Baselines

To demonstrate the efficiency and value of our RL agent, we will evaluate its performance against three distinct baselines:

1. **No-Acquisition Baseline:** A standard supervised classifier trained and tested solely on the base STATS19 features. This baseline incurs zero latency and zero API cost but represents the lower bound for prediction accuracy.
2. **Full-Acquisition Baseline:** A classifier that always acquires all external features A_1, A_2, and A_3)for every single accident record. This represents the upper bound for potential accuracy, but at the maximum possible cost and latency.
3. **Static Feature Priority Baseline:** A heuristic baseline where external sources are ranked globally in advance, producing a fixed priority order (e.g., Weather → OSM →

AADF). Given a cost budget equal to the average cost spent by the RL agent, the system acquires sources one by one following this pre-defined order until the budget is exhausted, then predicts using the corresponding pre-trained model. Unlike the RL agent, this baseline cannot adapt its acquisition strategy to the specific circumstances of each individual accident — it applies the same fixed order regardless of whether the current accident already has clear severity indicators in the base STATS19 features.

2.4 Extensions and Analysis

Beyond the core AFA framework, we include the following research components to deepen the analysis and strengthen the project's contributions.

Confidence Score

Instead of outputting only a severity class, the classifier will also output a calibrated uncertainty estimate (e.g., predicted class probabilities). This allows the dispatch system to flag low-confidence predictions for human review, which is particularly important for the Fatal and Serious classes where misclassification has the highest real-world cost.

Leakage Audit

We will train two models — one with all STATS19 features and one restricted to dispatch-time features only (i.e., those available within seconds of the 999 call) — and measure the performance gap. This quantifies the accuracy cost of real-world deployability — how much predictive performance we lose by restricting the model to only the features available at the moment of the 999 call, compared to using the full STATS19 feature set.

Ablation Study

The system trains 8 classifiers, one per possible combination of the three external sources (OSM, AADF, and Weather). We conduct a full ablation by evaluating all 8 models individually, directly measuring the independent and combined contribution of each external source to prediction accuracy. Within each model, random STATS19 field masking during training. The results also inform the calibration of the acquisition cost penalties C_i in the RL reward function — specifically, a source that produces a large accuracy gain relative to its API cost should receive a lower penalty, while a source that contributes little should receive a higher one. For example, if the ablation shows that Weather adds +0.10 Macro F1, setting $C_{Weather} = 0.15$ would cause the agent to never acquire it since the cost outweighs the benefit — a bad calibration. Setting

$C_{Weather} = 0.05$instead would reflect its true value and allow the agent to learn when acquiring it is worthwhile. The ablation thus gives a principled basis for setting the $C_i$values rather than picking them arbitrarily, ensuring the RL agent faces a meaningful trade-off rather than trivially always acquiring or always skipping each source.

Officer vs. Data Gap

STATS19 includes a subjective weather field filled by the attending officer after the fact. We will directly compare this categorical field against the objective Open-Meteo measurements at the same location and time, and test which source is more predictive of accident severity. This quantifies the value of replacing subjective officer assessments with objective sensor data.

Temporal Drift

We will train on earlier years of STATS19 data and test on more recent ones to examine whether accident severity patterns are stable over time.

3. Data Sources

3.1 Primary Dataset — STATS19 (UK Official Accident Database)

Source: UK Department for Transport, accessed via data.gov.uk (road-accidents-safety-data) and the ropensci/stats19 R package (github.com/ropensci/stats19).

STATS19 is Britain's official road traffic casualty database, recording every personal injury accident reported to the police since 1979. The dataset is divided into three linked tables joinable by accident index:

- Accidents table: location (lat/lon), date, time, road type, weather, light conditions, speed limit, junction type, severity
- Casualties table: age, sex, casualty class, injury severity per person
- Vehicles table: vehicle type, manoeuvre, driver age, engine capacity

3.2 Traffic Volume — DfT Annual Average Daily Flow (AADF)

Source: UK Department for Transport, roadtraffic.dft.gov.uk/downloads — file: dft_traffic_counts_aadf.zip.

This dataset provides the estimated number of vehicles passing each count point on the major road network on an average day of the year, covering years 2000 to 2024 with 578,217 records. Each count point has a geographic coordinate and road identifier.

Join method: For each accident, we identify the nearest count point within 500 metres using spatial nearest-neighbour matching (scipy.spatial.cKDTree). The AADF for the accident year is then attached as a feature. This produces a 'road busyness' feature entirely absent from the STATS19 data, capturing the exposure level of the road where the accident occurred.

3.3 Road Infrastructure — OpenStreetMap (Overpass API)

Source: OpenStreetMap, queried via the Overpass API (overpass-api.de). Free, no registration required.

For each accident location we perform a spatial query to extract road-level features that are not captured in STATS19:

- Speed camera within 500m (highway=speed_camera tag) — binary feature
- Distance to nearest speed camera in metres — continuous feature
- Number of lanes on the road at the accident point
- Presence of central median or road divider
- Presence of street lighting (highway=street_lamp within 100m)

3.4 Historical Weather — Open-Meteo Historical Weather API

Source: Open-Meteo, open-meteo.com/en/docs/historical-weather-api. Professional API plan required for historical data access.

STATS19 includes a categorical weather field (fine, raining, fog, etc.) recorded by the attending officer. However, this is a subjective assessment made at the scene — not an objective measurement. Open-Meteo's Historical Weather API provides high-resolution reanalysis weather data covering any location on earth at hourly resolution, going back to 1940. Key variables available:

- Temperature at 2m (°C) — hourly, at exact accident location
- Precipitation (mm/hour) — distinguishes drizzle from heavy rain
- Wind speed at 10m (km/h)
- Visibility (metres) — critical for accident severity
- Weather code — standardised WMO weather condition at the hour of accident
- Relative humidity (%) — contributes to road surface friction
- Snowfall and snow depth — particularly relevant for winter accidents

Join method: For each accident, a single API call is made with the exact accident latitude, longitude, date, and hour. Open-Meteo returns hourly weather for that specific point — no station proximity matching is required. This data is significantly more accurate than station-based approaches for the UK, which has many rural areas far from weather stations. API calls are batched and cached locally to minimise cost and request volume.

This source replaces the subjective categorical STATS19 weather field with objective continuous measurements at the exact location and time of each accident — a substantial qualitative improvement in weather feature quality.

4. System Description

The system is composed of eight components:

- **Data Acquisition:** Downloads all static dataset files (STATS19, DfT AADF) and performs all pre-training API calls (OpenStreetMap, Open-Meteo). During training, all external data is fetched once and cached locally. At inference time, the relevant API calls are made in real time by the RL agent.

- **Feature Engineering Pipeline:** Performs spatial and temporal joins. Output: a single fully-enriched feature table per accident record, from which any of the 8 external-source subsets can be derived. Per-feature acquisition costs are annotated at this stage.

- **Preprocessing Module:** Handles missing values, encodes categorical variables, and normalises continuous features. Fitted on the training set only and applied identically to validation and test sets to prevent leakage.
  Also generates the masked training data for the classifiers: for each record, multiple copies are created with random subsets of STATS19 fields masked, so each of the 8 classifiers learns to produce meaningful predictions and calibrated confidence estimates from any partial STATS19 input.

- **Classifier Training Module:** Trains all 8 supervised classifiers — one per possible subset of the three external sources (OSM, AADF, Weather). Each classifier is trained with random masking of individual STATS19 fields, enabling it to handle any partial STATS19 input while knowing exactly which external sources are present. All 8 models are trained offline before RL training begins and remain frozen during RL training.
- **Evaluation & Experiment Runner 1:** Evaluates all 8 classifiers and runs all pre-RL analyses: the ablation study across all 8 external-source subsets, the leakage audit, the officer vs. data gap experiment, counterfactual analysis ,and the temporal drift analysis.

- **RL Agent Training Module:** Trains the Active Feature Acquisition policy using the pre-trained classifiers as a frozen environment. Each episode corresponds to one accident record. The agent observes the current state s_t, selects acquisition or termination actions, and receives the composite reward at the end of the episode.

- **Evaluation & Experiment Runner 2:** Evaluates the trained RL agent against all 3 baselines (No-Acquisition, Full-Acquisition, Static Priority), and generates comparison tables and plots.

- **Inference Pipeline:** Receives a live accident report, initialises the state with automatically available STATS19 fields, and runs the trained RL policy — querying individual STATS19 fields or making real API calls as the agent directs. After each acquisition, the classifier corresponding to the current external-source subset is re-run on the updated features to recompute p_t. When the agent terminates, the current p_t is returned as the final predicted severity distribution, and the argmax is reported as the predicted class with its associated confidence score.

5. Data Sources Summary

Source	URL	Role	Join Method	Cost
STATS19 (DfT)	https://github.com/ropensci/stats19	Primary dataset + labels	Primary (no join)	Free
DfT AADF Traffic	roadtraffic.dft.gov.uk/downloads	Traffic volume feature	Spatial (nearest count point)	Free
OpenStreetMap	overpass-api.de	Road infrastructure features	Spatial (lat/lon radius)	Free
Open-Meteo Historical	open-meteo.com/en/docs/historical-weather-api	Hourly weather at exact location	API call by lat/lon + datetime	Paid (Professional plan) €99